Received: October 18, 2021. Revised: December 20, 2021.

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Modular Battery Management System with Machine Learning-Driven State-of-Health Estimation: Design, Implementation, and Comparative Analysis for Electric Vehicle Applications

Author' Madhuri Pagale [1] **Author's Ashwini Deshpande [2]**

Author' Arnav Patel [3] **Author's Nirbhik Navin [4]** **Author's Aditya Pokharana [5]**

[1]*Pimpri Chinchwad College of Engineering, CSE (AI&ML Dept), India, madhuri.pagale@pccoepune.org*
[2]*Pimpri Chinchwad College of Engineering, CSE (AI&ML Dept), India, ashwini.deshpande@pccoepune.org*
[3]*Pimpri Chinchwad College of Engineering, CSE (AI&ML Dept), India, patelarnav1019@gmail.com*
[4]*Pimpri Chinchwad College of Engineering, CSE (AI&ML Dept), India, nirbhiknavin@gmail.com*
[5]*Pimpri Chinchwad College of Engineering, CSE (AI&ML Dept), India, adityapokharna28@gmail.com*

Abstract: The efficient monitoring and intelligent management of lithium-ion battery packs are imperative for electric vehicle (EV) performance, safety, and longevity. This paper introduces a full-fledged solution integrating hardware and software design of a modular Battery Management System (BMS) and machine learning (ML) models for State of Health (SOH) estimation. The proposed BMS is a master-slave architecture system consisting of LTC6812 ICs for cell-level monitoring and an STM32F446RE microcontroller as a master unit. The system permits real-time State of Charge (SOC) estimation, thermal and current management, capacity tracking, and cell balancing under rigorous communication protocols.

A predictive comparison of the four machine learning algorithms that we use-Random Forest (RF), XGBoost (XGB), Support Vector Machine (SVM), and Gradient Boosting Regression (GBR).-is done against real battery data from the field. The results show that tree-based models outperform other models with GBR giving a significant contribution to the state of health prediction. It makes a case for the integration of ML-based diagnostics into real-time BMS for better battery reliability, predictive maintenance, and performance optimization of the electric vehicle.

Keywords—Battery Management System, Machine Learning, State of Health, Electric Vehicles, SOC Estimation.

1. Introduction

The recent rise of electric vehicles (EVs) as an option for sustainable vehicular alternatives stems from their much lower emissions and energy efficiency compared to internal combustion engine vehicles. On the other hand, at the core of the performance of any electric vehicle is an energy storage system in the form of a lithium-ion battery pack - a form of energy storage very compact and favored for having a high energy density, life cycle, and energy efficiency [1]. But for their safety, longevity, and good performance, an intelligent and reliable Battery Management System has to be mounted onboard.

The BMS is such that it supervises and monitors real-time important battery parameters such as voltage, current, temperature, and state of charge (SOC). And, it conserves all the roles indicating protection against inappropriate operating conditions, prevention of unwanted

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

charge/discharge cycles, and balancing the cells at the end. Last but not least, the modern BMS must provide predictive diagnostics for evaluating the future degradation of the battery under consideration.

The most important parameter for determining the state of health of a battery is "State of Health" (SOH)-a figure that defines the remaining useful life and capacity of a battery compared to the baseline. Having reliable SOH estimations helps to detect faults earlier, helps to optimize charge strategies, and aids in better long-term planning either for EV performance or maintenance. Old methods of estimating SOH depend on electrochemical or physics-based models, but the non-linearity and data dependence of real-world battery degradation have made it difficult for such models.

Machine Learning (ML) has emerged as a very good capability to understand and model complex, non-linear behavior in battery systems; hence it is a data-driven approach. The idea is that the ML model more accurately predicts SOH by learning from charge/discharge data from reality, possibly even without knowing the chemistry of the battery in detail. In this work, we will study and compare four machine learning model frameworks-Random Forest (RF), XGBoost (XGB), Gradient Boosting Regression (GBR), and Support Vector Machine (SVM)-using experimental battery datasets for finding the best among them in terms of SOH prediction accuracy.

This research addresses two key aspects:

1. BMS Hardware & Software Implementation creating a modular and scalable Master-slave BMS architecture for more monitoring and control.
2. Machine Learning Based SOH Estimation – Comparison of different ML models to find the most effective ones for accurate SOH prediction.

2. Literature survey:

Battery Management Systems (BMS) are crucial for optimizing lithium-ion battery safety and performance. Hu et al. (2017) outlined essential BMS functions such as SOC estimation and thermal control. Shang et al. (2021) enhanced BMS performance using STM32 microcontrollers with FPGA/DSP, though at the cost of complexity.
For SOH estimation, traditional electrochemical models offer interpretability but struggle with real-world accuracy. Machine learning (ML) has emerged as a superior alternative, with Waag et al. (2014) and Yang et al. (2020) showing improved prediction using methods like XGBoost. Zhang et al. (2021) used hybrid features for greater precision, though computationally intensive.
Deep learning approaches like LSTM (Sun et al., 2021) offer long-term degradation tracking, but hardware constraints limit real-time use. Communication protocols and fault detection models (Li et al., 2023; Qian et al., 2021) further enhance BMS robustness.
Overall, integrating ML with efficient BMS architecture holds promise for accurate, scalable SOH estimation and real-time diagnostics.

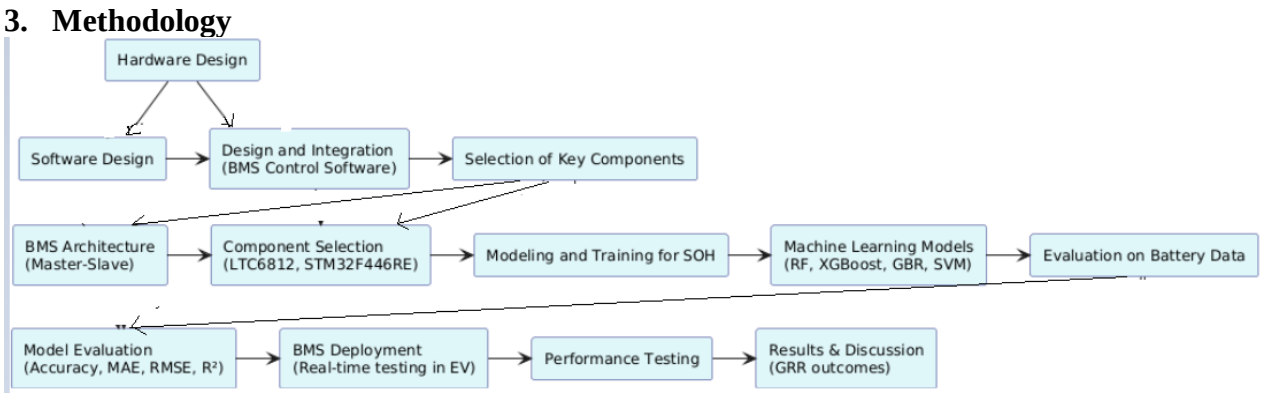
Table 1: Summary of Key Studies on Battery Management Systems and Health Estimation

Sr. No	Title	Author(s)	Year	Method	Approach	Pros/Cons
1	Battery Management System: An Overview on its Application in the Smart Grid and Electric Vehicles	X. Hu, L. Xu, M. Liu, S. Di	2017	Overview of BMS	Overview of BMS functions (SOC estimation, thermal management, energy optimization)	Pros: Comprehensive overview of BMS roles; Cons: Lacks implementation details
2	Development and Implementation of an Intelligent BMS Based on FPGA and DSP	L. Shang, F. Hu, R. Xie, Y. Ding	2021	FPGA & DSP-based BMS	Integration of STM32-based microcontrollers in BMS	Pros: Practical implementation details; Cons: Limited scalability discussion

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

3	Critical Review of SOH Estimation Methods for Lithium-Ion Batteries	W. Waag, C. Fleischer, D. U. Sauer	2014	Comparative Review	Compared electrochemical vs ML-based SOH estimation	Pros: Detailed comparative analysis; Cons: Lacks practical validation
	Machine Learning Approach for SOH Estimation Using XGBoost	Y. Yang, Z. Rao, J. Liu	2020	Machine Learning	XGBoost for battery degradation prediction	Pros: High prediction accuracy; Cons: Requires extensive dataset
5	Hybrid Features for Battery SOH Estimation	C. Zhang, K. Li, P. Wang	2021	Hybrid Model	Voltage relaxation & Coulomb counting for SOH tracking	Pros: Improved SOH estimation; Cons: Complex feature extraction
6	Mathematical Modeling of Lithium Batteries	R. E. White, R. J. Kee, J. Newman	1991	Electrochemical Modeling	Models lithium-ion degradation mathematically	Pros: Strong theoretical foundation; Cons: Lacks real-world validation
7	Hybrid ML & Electrochemical Model for Lithium-Ion Battery Prognostics	H. Kim, H. C. Kim, C. B. Song	2022	Hybrid Model	ML and electrochemical modeling combined	Pros: Increased prediction accuracy; Cons: Computationally expensive
8	Deep Learning-Based SOH Estimation Using LSTM	S. Sun, W. Dong, F. Gao	2021	Deep Learning	LSTM networks for long-term SOH tracking	Pros: Effective for time-series data; Cons: Requires large dataset
9	Review on Deep Learning for Battery Health Management	Y. Chen, H. Fang, X. Zhang	2022	Review Paper	Case studies on real-world SOH prediction using deep learning	Pros: Real-world validation; Cons: Limited coverage of different architectures
10	Data-Driven and Model-Based Battery Fault Diagnosis	J. Li, H. Yang, Y. Liu	2023	Fault Diagnosis	Real-time fault detection in BMS	Pros: Practical approach for diagnostics; Cons: Limited fault types covered
11	ISO-SPI Based High-Speed Communication for BMS	K. Qian, Z. Wang, T. He	2021	Communication Protocol	Optimized isoSPI protocol for BMS	Pros: Faster and efficient data transfer; Cons: Hardware-dependent

3. Methodology



(Fig1. BMS Design with ML Integration)

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

The key steps involved in the design and implementation of a **Battery Management System (BMS)** integrated with **Machine Learning (ML)** for **State of Health (SOH)** estimation of an electric vehicle's battery pack. It consists of the following major components:

1. **Hardware Design**: This step involves designing and integrating the physical components of the BMS. The system follows a **Master-Slave architecture**, where the **master controller** (STM32F446RE) communicates with **slave modules** (LTC6812) to monitor and manage the battery cells. Key components are selected for their performance, power efficiency, and scalability.
2. **Software Design**: The software for the BMS manages the control algorithms for **State-of-Charge (SOC)** estimation, fault detection, and real-time monitoring. It includes integrating **Machine Learning models** (Random Forest, XGBoost, Gradient Boosting Regression, and Support Vector Machine) to predict the **State of Health (SOH)** based on real-world battery data.
3. **Component Selection**: Critical components like **LTC6812 ICs** for cell monitoring and **STM32F446RE** microcontrollers for processing and communication are chosen based on their capabilities in handling battery data, minimizing errors, and enabling future expansion of the system.
4. **ML Models**: The BMS uses machine learning models to predict the SOH of the battery pack, which helps in diagnosing battery degradation and forecasting remaining battery life. Four models were tested: **Random Forest**, **XGBoost**, **Gradient Boosting Regression (GBR)**, and **Support Vector Machine (SVM)**.
5. **Model Evaluation**: The models are evaluated based on **accuracy**, **MAE**, **RMSE**, and **R^2 score**. **Gradient Boosting Regression (GBR)** was found to perform best, offering the most accurate SOH prediction, followed by **Random Forest** and **XGBoost**. **SVM** performed poorly in this context.
6. **BMS Deployment**: After evaluating the models, the **best-performing ML model** (GBR) is deployed into the BMS, enabling real-time **SOH prediction** and **battery diagnostics** on the vehicle.
7. **Results & Discussion**: The final section discusses the results of the testing, confirming **GBR** as the most effective model for SOH prediction and highlighting areas for future improvements, such as expanding the dataset and implementing deep learning models like **LSTM** for better long-term accuracy.

3.1 Hardware:

This project focuses on developing a Battery Management System (BMS) for a Formula Student Electric (FSAE)[16] vehicle and predicting the State of Health (SoH) of its battery pack. The system was designed with reference to various sources, with significant contributions from Francesco Rizzotto's (Rizotto, 2021/2022)work on BMS architecture.

3.1.1 Battery Pack Configuration

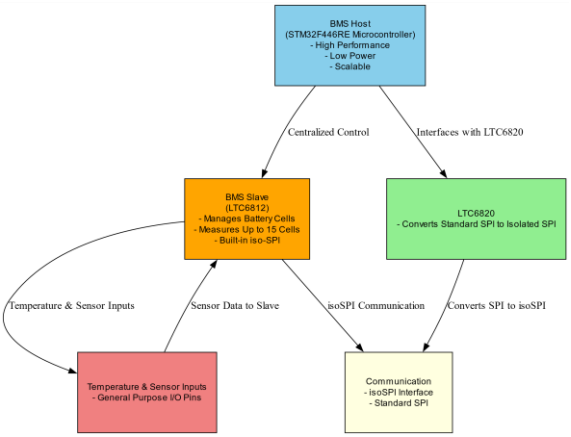
The battery pack of the vehicle uses a 112s2p configuration with Malesta cells (Melasta), consisting of two 20s2p modules and four 18s2p modules. This configuration was selected based on the vehicle’s requirements, including the necessary voltage and current specifications for optimal performance.

3.1.2 System Architecture

- The BMS employs a centralized architecture[18], consisting of two main components: the host and the slave modules. The host uses the LTC6820 (Devices) integrated circuit, which interfaces with the microcontroller and converts standard SPI communication into isolated SPI[17]. On the slave side, the LTC6812 (devices)is used to manage the battery cells. Key features of the LTC6812 include:
- Measurement of up to 15 battery cells in series
- 16-bit ADC with programmable noise filtering
- General Purpose I/O pins for temperature and other sensor inputs
- Built-in iso-SPI interface
- Operating temperature range from -40°C to +125°C

The STM32F446RE (Microelectronics) microcontroller was selected for its high performance, community support, low power consumption, and scalability. These factors make it an excellent choice for the BMS master unit, ensuring efficient and reliable monitoring and management of the electric vehicle's battery system while allowing for future expansion and optimization

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/



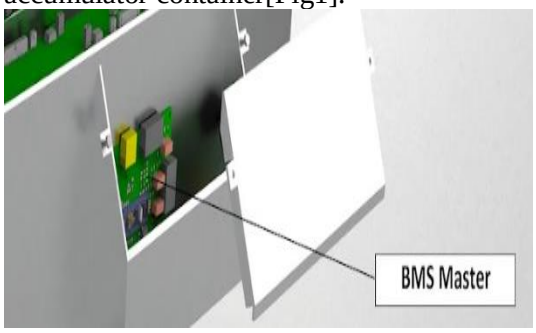
(Fig2. BMS Host and Slave Communication)

3.1.3 Hardware Design

Hardware design considerations were categorized into the master and slave configurations:

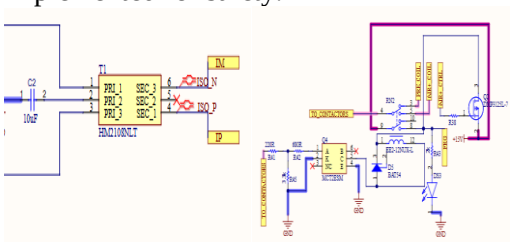
Master PCB Design:

- Dimensions of the master PCB were selected based on its placement in the accumulator container[Fig1].



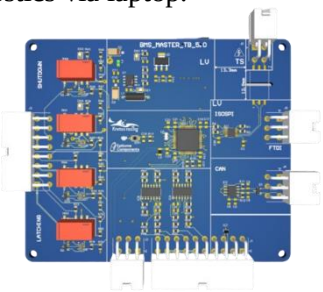
(Fig3.Placement of BMS in the Battery Pack)

- Ensured minimal harness complexity in the high-voltage section.
- Inductive and galvanic isolation between high-voltage and low-voltage systems was implemented for safety.



(Fig4.Use of Pulse transformers and Optocouplers for isolation)

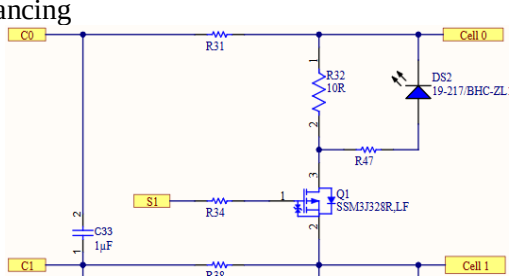
- Integrated precharge circuitry and shutdown functionality for safety during operation.
- Data logging capabilities for real-time analysis were incorporated, along with ESD protection diodes for communication lines.
- F232 USB UART IC (chip, FT232R USB UART IC) integration was included for BMS diagnostics via laptop.



(Fig5.BMS Master)

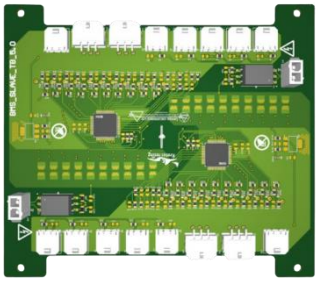
Slave PCB Design:

- The slave configuration was adapted to monitor cells in a 9-cell or 10-cell configuration using two ASICs for fault monitoring, communication, and fast data processing.
- External MOSFET were used for cell balancing



(Fig6.External MOSFET used for Cell balancing)

- PCB design ensured separate traces for power and measurement signals to avoid interference.



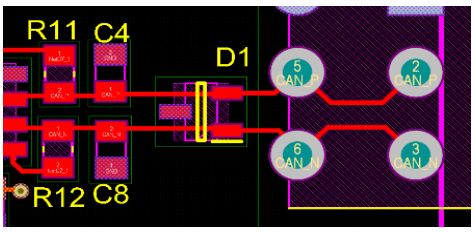
(Fig7.BMS Slave)

3.1.4 PCB Design

Multilayer PCB designs were employed to separate analog, digital, and communication layers. Design techniques included:

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

- Shielding signals using ground and power polygons.
- Decoupling components were placed close to ICs for stability.
- Differential pair routing for communication lines.



(Fig8.Differential pair routing of CAN)

- Careful component layout based on datasheet recommendations to minimize noise and signal interference.

	Top Overlay		Overlay				
	Top Solder	Solder Resist	Solder Mask		0.5mil	3.5	
1	Top Layer	CF-004	Signal	2oz	2.756mil		
	Dielectric 2	PP-006	Prepreg		3.248mil	4.1	0.02
2	Layer 2	CF-004	Signal	1oz	0.7mil		
	Dielectric 3	PP-001	Prepreg		4mil	3.9	0.02
3	Layer 3	CF-003	Signal	1oz	1.378mil		
	Dielectric 1	FR-4	Dielectric		71.416mil	4.8	
4	Layer 4	CF-003	Signal	1oz	1.378mil		
	Dielectric 4	PP-001	Prepreg		4mil	3.9	0.02
5	Layer 5	CF-004	Signal	1oz	0.7mil		
	Dielectric 5	PP-006	Prepreg		3.248mil	4.1	0.02
6	Bottom Layer	CF-004	Signal	2oz	2.756mil		
	Bottom Solder	Solder Resist	Solder Mask		0.5mil	3.5	
	Bottom Overlay		Overlay				

(Fig9.Multilayer PCB design using Altium)

3.2 Software Design

The software for the Battery Management System (BMS) was developed to ensure robust battery pack management and efficient data communication. Key considerations include:

- **Operating Modes:** The BMS operates in multiple modes, including Charge, Discharge, Configuration, and Diagnostic modes [15].
- **State-of-Charge (SOC) Estimation:** An SOC estimation algorithm predicts the remaining charge based on historical and real-time data [7].
- **Fault Detection:** The system generates fault codes to detect anomalies within the battery pack [8].
- **Data Communication:** Critical battery parameters are transmitted via the CAN bus, ensuring real-time monitoring [11].
- **Real-Time Logging:** Key metrics such as SOC, pack voltage, and temperature are logged continuously for performance tracking [15].

Model Architecture & Mathematical Formulation

1. XGBoost (Extreme Gradient Boosting)

XGBoost iteratively builds regression trees to correct residuals from prior models. It incorporates L1/L2 regularization to penalize complex trees and a second-order Taylor approximation for efficient loss optimization. Shrinkage (learning rate) scales tree contributions, preventing overfitting [10].

- **Mathematical Formulation**:

Regularized Objective Function:

$$L(\theta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \, ||\theta||^2$$

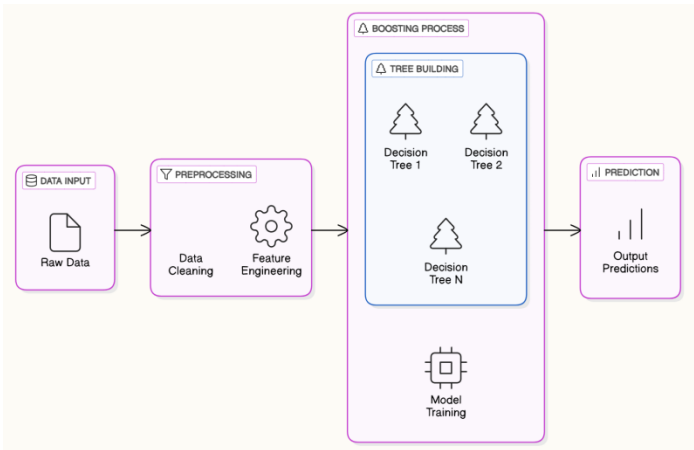
Where λ controls regularization strength, and $\theta\theta$ represents model parameters.

Additive Training:

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta f_t(x)$$

Where η is the learning rate, and $f_t(x)$ is the tth tree.

Structure Score:
Splits are evaluated using gain metrics derived from first- and second-order gradients.



(Fig10. XGBoost Architecture)

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Received: October 18, 2021. Revised: December 20, 2021.

2. Gradient Boosting Regression (GBR)

GBR sequentially trains weak learners (typically decision trees) to minimize residual errors. Unlike XGBoost, it lacks explicit regularization but refines predictions through gradient descent in function space [12].

- **Mathematical Formulation**:

Additive Model:

$$F_m(x) = F_{m-1}(x) + \gamma h_m(x)$$

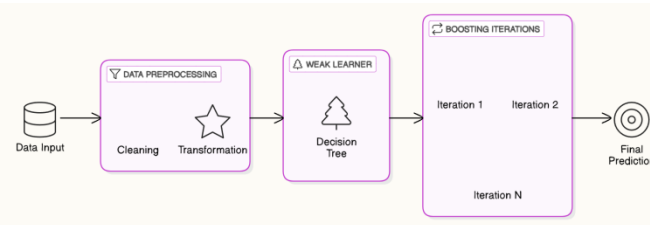
learner, and γ is the learning rate.

Pseudo-Residuals:

$$r_{im} = y_i - F_{m-1}(x_i)$$

Used to fit the mth tree.

Loss Minimization: The learning rate γ is optimized via line search.



(Fig11. GBR Architecture)

3. Random Forest (RF)

Random Forest (RF) employs an ensemble of decision trees trained via bootstrap aggregation (bagging). Each tree is constructed using a random subset of the training data (with replacement) and a random subset of features at each split. Predictions are aggregated through majority voting (classification) or averaging (regression), mitigating overfitting and enhancing stability [9].

- **Mathematical Formulation**:

Ensemble Prediction:

$$\hat{y}(x) = \frac{1}{N}\sum_{i-1}^{n} h_i(x)$$

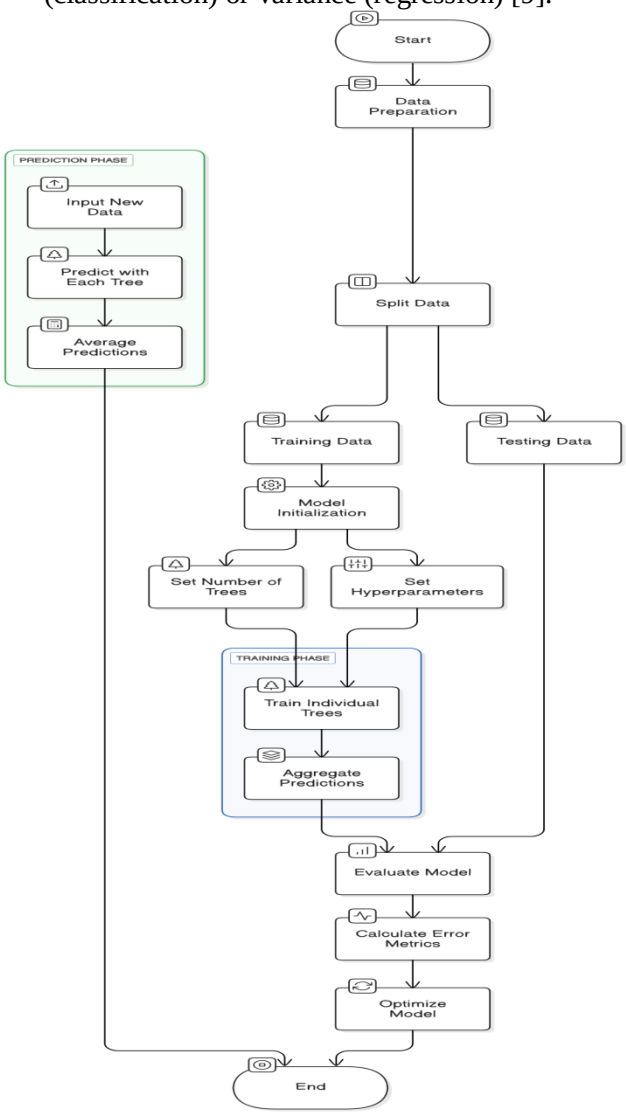
Where $h_i(x)$ is the prediction of the ith tree, and N is the total number of trees.

Regression Splitting Criterion (Mean Squared Error):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where n is the number of samples at a node.

Feature Randomness: At each split, m features ($m \leq M$, M=total features) are evaluated to minimize Gini impurity (classification) or variance (regression) [9].



(Fig12. Random Forest Architecture)

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

4. Support Vector Machine (SVM)

SVM identifies a hyperplane $w^T x + b = 0$ that maximizes the margin between classes (classification) or minimizes deviations within $\epsilon\epsilon$ (regression). Kernel functions (e.g., RBF, polynomial) map data to higher dimensions for nonlinear separability [13].

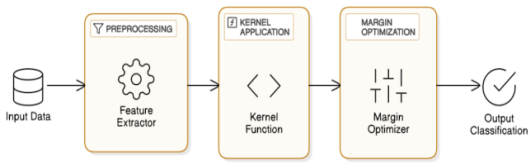
Mathematical Formulation:

- **Margin Maximization**:

$$min_{w,b} \frac{1}{2} \left|\left|w\right|\right|^2 \; subject\ to\ \ y_i(w^T x_i + b) \geq 1\,, \forall_i$$

- **Kernel Trick**: For nonlinear cases, replace $x_i^T x_j$ with (x_i, x_j) , where K is a kernel function.
- **Regression (SVR)**: Uses $\epsilon\epsilon$-insensitive loss:

$$L_\epsilon\,(y, \hat{y}) = (0, |y - \hat{y}| - \epsilon\,)$$



(Fig. 13 SVM Architecture)

4.2.2 **Model Training & Evaluation Metrics:**

- **Mean Absolute Error (MAE)**:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

- **Root Mean Squared Error (RMSE)**:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **R^2 Score**:

$$R^2 = \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y}_i)^2}$$

Table 2: Performance Comparison

Model	MAE	RMSE	R^2 Score
Random Forest	0.0358	0.6133	0.9650
XGBoost	0.0411	0.6549	0.9601
Gradient Boosting	0.0331	0.5714	0.9696
SVM	0.6786	3.1439	0.0805

5. Results & Discussion:

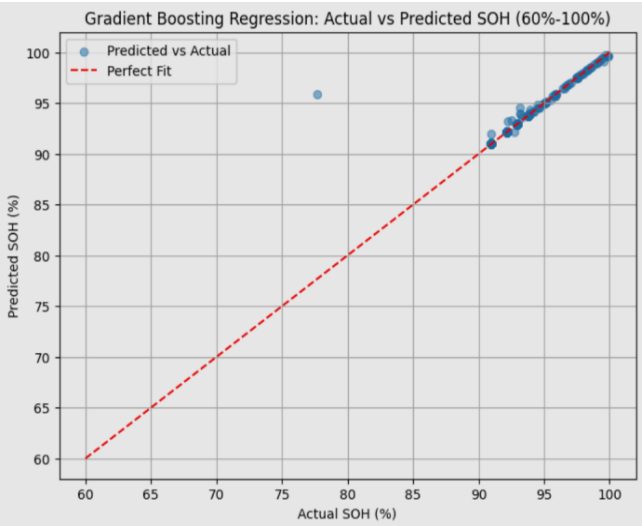
Analysis:

Tree-Based Models (RF, XGBoost, GBR): Achieved high R^2 scores (>0.96), indicating strong predictive power. GBR's iterative residual correction likely contributed to its superior RMSE.

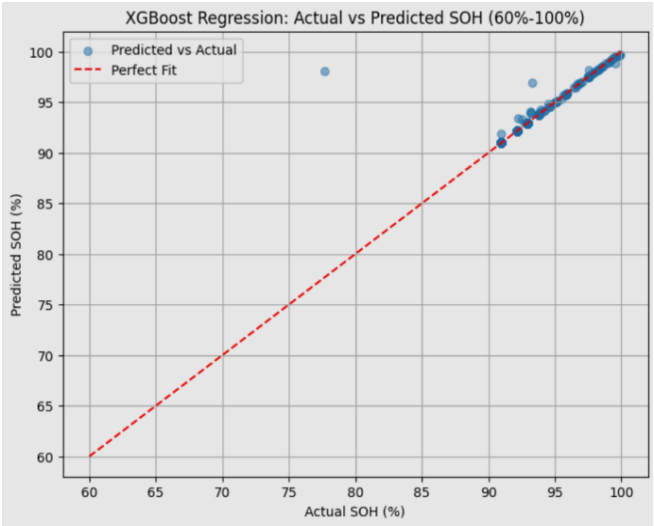
SVM: Poor performance (R^2=0.08) suggests incompatibility with the data's nonlinearity or inadequate hyper parameter tuning.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

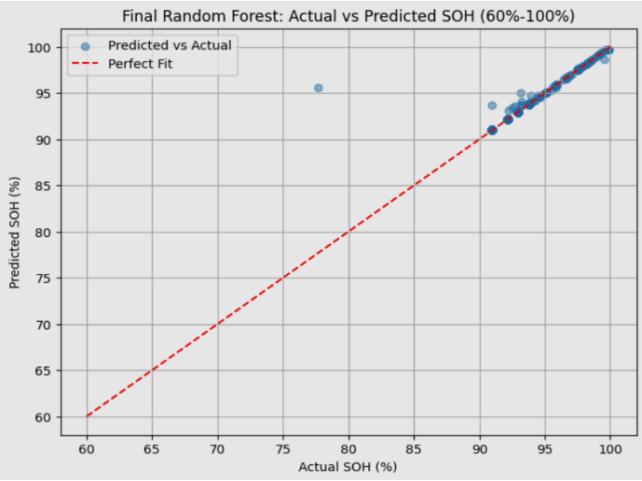
Received: October 18, 2021. Revised: December 20, 2021.



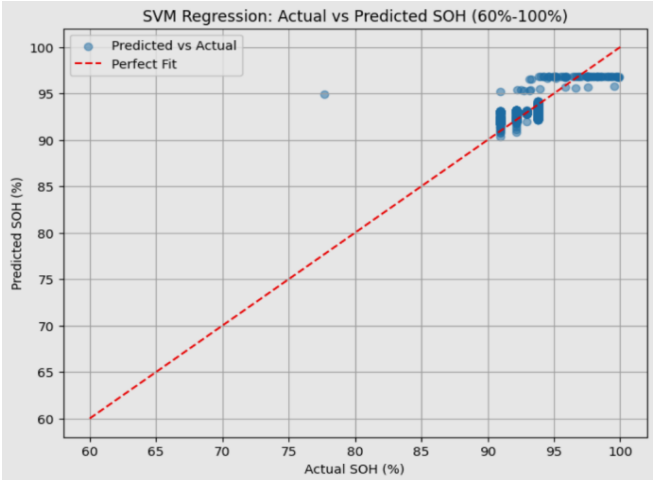
(a)GBR Results



(b)XGBoost Results



(c)Random Forest Results



(d)SVM Results

(Fig14.Results (a)GBR Results (b)XGBoost Results (c)Random Forest Results (d)SVM Results)

5.1 **Analysis of Prediction Graphs:**

The Actual vs. Predicted SOH plots for all four models are shown in the figure above. The red dashed line represents the ideal case where predictions perfectly match actual values.

Gradient Boosting Regression (GBR): Most of the predicted SOH values align closely with the perfect fit line, confirming the model's superior performance.
Random Forest & XGBoost: These models also show high prediction accuracy, with minimal deviation from the actual values.
Support Vector Machine (SVM): The SVM model exhibits large errors, especially for lower SOH values, with a wider spread and multiple outliers. This explains its high RMSE (3.1439) and low R^2 (0.0805).

5.2 **Key Observations**

- GBR is the best-performing model, followed closely by Random Forest and XGBoost.
- SVM is not suitable for SOH prediction, as **Future Improvements**
- Expand the dataset to further improve model generalization.
- Test deep learning approaches (LSTM, ANN) for time-series SOH estimation.
- Deploy the best-performing model (GBR) in a real-time BMS system for further validation.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

5.4 Conclusion

This study integrates advanced hardware and machine learning to enhance EV battery management. The modular BMS, using LTC6812 ICs and an STM32F446RE controller, ensures scalable performance with safety features like galvanic isolation and isoSPI/CAN protocols. Among the ML models tested, Gradient Boosting Regression (GBR) achieved the highest SOH prediction accuracy (MAE: 0.0331, R^2: 0.9696), proving highly effective for real-time diagnostics. While Random Forest and XGBoost also performed well, SVM underperformed, highlighting limitations of non-ensemble models. This fusion of reliable BMS hardware and ML-driven diagnostics promises longer battery life, proactive maintenance, and enhanced safety. Future directions include adapting to various chemistries and deploying lightweight neural networks for edge computing in EVs.

Future Work:

1. Deep Learning Integration for SOH Prediction: Incorporating LSTM networks for capturing temporal dependencies in battery degradation behavior. Comparing LSTM-based architectures against classical ML models in order to quantify improvements in SOH estimation accuracy/generalization ability.

2. Real-Time Implementation in BMS Firmware: Importing the ML-based SOH estimation models directly into the BMS firmware for deployment.Performance evaluation of data-driven SOH prediction methods under real-time onboard conditions.

3. Hybrid ML-Physics-Based Modeling: Exploring hybrid approaches that combine Machine Learning models with electrochemical and physical models of degradation to enhance long-term prediction accuracy. Implementing feature-engineering techniques that exploit battery impedance, resistance, and electrochemical properties to enhance SOH tracking.

4. Increasing Data Acquisition and Generalization of the Model: To acquire more naturalistic-degradation battery datasets from different sources to improve model robustness. Transfer learning and domain adaptation techniques will be investigated for improving generalizations over different battery chemistries and usage scenarios.

6. REFERENCES:

[1] (Rizotto, 2021/2022) Link: Battery Management System for a Formula SAE car concept RIZZOTTO, FRANCESCO 2021/2022

[2](Melesta)Link:https://www.melasta.com/web/userfiles/download/SLPBB042126.pdf

[3] (Devices) URL: LTC6820 datasheet

[4] (Microelectronics) URL: STM32F446RE datasheet

[5] (devices) URL:LTC6812 datasheet

[6] (chip, FT232R USB UART IC) FTDI datasheet

[7] Plett, G. L. (2004). Extended Kalman Filtering for Battery Management Systems. Link: ScienceDirect

[8] Dey, S., et al. (2019). Model-Based Fault Diagnosis of Li-Ion Batteries. Link: IOP Science

[9] Breiman, L. (2001). Random Forests. Link: Springer

[10] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Link: arXiv

[11] Zheng, Y., et al. (2018). Real-Time CAN Bus-Based BMS for EVs. Link: IEEE Xplore

[12] Friedman, J. H. (2001). Gradient 14Boosting Machine. Link: JSTOR

[13] Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. Link: Springer

[14] Willmott, C. J., & Matsuura, K. (2005). Advantages of MAE Over RMSE. Link: Int-Resp

[15] Tran, M. K., et al. (2018). A Comprehensive Review of Battery Management Systems. IEEE. Link: IEEE Xplore

7. Author Contributions

"Conceptualization, Madhuri, Ashwini, Arnav, Nirbhik, and Aditya; Methodology, Madhuri and Ashwini; Hardware, Nirbhik and Aditya; Software, Arnav; Validation, Arnav; Data Curation, Aditya, Nirbhik, and Arnav; Writing—Original Draft Preparation, Madhuri, Ashwini, Arnav, Nirbhik, and Aditya; Writing—Review and Editing, Arnav and Madhuri; Visualization, Arnav; Supervision, Madhuri and Ashwini; Project Administration, Madhuri and Ashwini; Funding Acquisition, Madhuri and Ashwini.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/